Arquitectura del compilador de Compiscript

Un único núcleo compila los bytes fuente a TAC determinista y a metadatos estáticos de generación de código. La CLI y HTTP son adaptadores; el IDE nunca implementa otro analizador sintáctico, analizador semántico ni formateador de TAC.

Flujo y responsabilidades

```
.cps bytes
  -> frontend.Parse (ANTLR CST -> project AST + frontend diagnostics)
  -> semantic.Analyze (names/types/scopes + semantic diagnostics)
  -> tac.Generate (only when preceding diagnostics are empty)
  -> tac.BuildCodegenReport (only after successful generation)
  -> CompilationReport {tac, diagnostics, scopes, codegen?}
```

El punto de entrada es Compile. El análisis sintáctico y el semántico se ejecutan una sola vez cada uno. Los diagnósticos del frontend y semánticos se ordenan de forma estable por ubicación, fase, código y mensaje, igual que en Analyze. Los errores de traducción (lowering) rechazan el resultado completo: no se filtra TAC parcial ni reporte de codegen. Los diagnósticos incluyen fase/código de tipo, mensaje y ubicación en la fuente; los AST mal formados o recuperados y las violaciones conservadoras de inicialización o de ámbito pueden reportar `TAC_UNSUPPORTED` en `codegen`, incluso en código inalcanzable. Ningún tipo de construcción válido del AST se rechaza solo por ser print, sin tipo o try/catch.

Los bytes fuente, el AST y las instantáneas de ámbitos semánticos son entradas, no almacenamiento mutable de codegen. La traducción deriva metadatos privados de enlace y disposición, y el reporte extendido une las identidades semánticas con el almacenamiento emitido. La aceptación con casos mixtos comprueba los hashes de las fuentes, las instantáneas del AST y de los ámbitos, que los resultados de Compile repetidos sean deterministas y la paridad de la fachada pública.

Componente	Responsabilidad / fuente
Frontend	frontend: diagnósticos del lexer/parser y mapeo al AST del proyecto; no traduce desde los tipos CST generados
Semántica	semantic: tipos, ámbitos, mutabilidad, declaraciones nominales y límites estáticos
IR/traducción	tac: instrucciones explícitas, asignación de temporales, transferencias de control, llamadas, entornos retenidos, objetos/listas
Fachada Compile	compile.go: flujo atómico y reporte de compilación aditivo
Fachada Analyze	analyze.go: contrato de reporte solo de análisis, sin cambios
CLI	compiscript-tac: texto/JSON y política de códigos de salida; compiscript: Analyze heredado
HTTP / navegador	servidor, web: una sola llamada a Compile, `Program.String()` del servidor como `tacText`, vistas seguras para texto
Prueba de ejecución	Oráculos acotados de marcos, referencias y entornos, solo para pruebas, en las pruebas del compilador; no hay VM de producto

IR y almacenamiento de un vistazo

La referencia TAC es la fuente autorizada para los campos de las instrucciones y los ejemplos. Las operaciones numéricas amplían explícitamente los enteros con `i2f`; las comparaciones, la negación y las ramificaciones modelan condiciones en cortocircuito. Las etiquetas y los saltos implementan los ciclos, el switch booleano y break/continue hacia el destino más cercano. Las llamadas y los retornos usan marcos nuevos; las llamadas indirectas usan closures retenidos seleccionados y guardas de firma. Las instrucciones de objetos asignan referencias, acceden a campos y enlazan métodos; las instrucciones de listas asignan, leen la longitud interna y realizan lecturas/escrituras indexadas. Las cadenas, funciones, objetos y listas son valores de referencia tipados, no codificaciones numéricas.

T9 agrega `print`, residuo, `declare` con celda undefined, uniones perezosas de ternarios, asignaciones que producen valor y almacenamiento/llamadas con valor null. T10 agrega operaciones verificadas sobre valores desconocidos, acceso y llamadas dinámicas a miembros y manejadores síncronos. Con esto se completa el despacho de

los 18 tipos de sentencia y 14 de expresión de ast/nodes.go; los nodos de recuperación `BadStmt`/`BadExpr` siguen rechazándose.

El marcador semántico de información ausente `TypeError` no cambia; el TAC protegido privado transporta las etiquetas reales de tiempo de ejecución sin agregar `TypeAny`. Un **campo sin anotación siempre conserva un contrato de almacenamiento desconocido**, incluso con un inicializador null o numérico; sin inicializador queda undefined. Las anotaciones explícitas siguen imponiendo la ampliación de tipos, las identidades nominales, las formas de las listas y las firmas completas de los invocables antes de los efectos del cuerpo. La ausencia de anotación de resultado en una función sigue significando null, no tipos de retorno inferidos. Los enlaces de catch usan almacenamiento de referencia de excepción y pueden ser capturados.

Espacio / identidad	Significado estático
`s<scopeId>.<name>`	Identidad semántica del enlace, no una dirección en bytes
`globals`	Celdas de enlace efectivamente emitidas que no pertenecen a un marco ni a un entorno, incluidos los enlaces del bloque principal
`main`	Celdas temporales emitidas; main no tiene un marco de activación inventado
`frame:<procedure>`	Continuación, enlaces de control y de acceso a globales, resultado opcional, receptor/parámetros, locales y temporales
`environment:s<scopeId>`	Disposición de enlaces léxicos retenidos y disposición del padre; cada entrada en tiempo de ejecución es una instancia distinta
`instance:<class>`	Desplazamientos de campos heredados y propios; los métodos se asocian a etiquetas de procedimiento
Etiquetas / ID de temporales	Las etiquetas de control, procedimiento y disposición no son celdas; las celdas `t0...` se reciclan tras su consumo final

`codegen` expone los símbolos semánticos, todas las ubicaciones reales, los espacios/ranuras y las etiquetas. Los desplazamientos y tamaños son **celdas abstractas**, no bytes ni direcciones físicas. Una celda de referencia no describe el tamaño del contenido en el heap. Un parámetro o local puede tener a la vez una ranura de preparación en el marco y un enlace en un entorno retenido; los campos heredados pueden aparecer en varias disposiciones de instancia. Un símbolo no emitido o que no es de almacenamiento no recibe una asignación inventada. Un ID de disposición estática nunca es la dirección de una invocación u objeto vivo.

Decisiones prácticas y sus razones

Decisión	Razón y límite
Un solo flujo y reporte aditivo	Evitar compiladores divergentes y preservar la compatibilidad de Analyze/TAC; derivar las disposiciones del estado emitido real en lugar de un segundo asignador
Reutilización de temporales por ranura libre más pequeña	Liberar tras el último consumidor emitido; acotar la demanda entre sentencias y conservar los valores vivos a través de llamadas con efectos. La demanda máxima no es una asignación de registros globalmente óptima
Celdas léxicas mutables retenidas	Los closures comparten la identidad del enlace, sobreviven al retorno de la función que los define y separan invocaciones distintas; no se copian instantáneas de escalares capturados
Un solo mecanismo de llamada/marco	Los métodos enlazados y los closures reutilizan la maquinaria de invocación, resultado y guardas en lugar de inventar un intérprete de objetos
Política de inicialización A	Decisión explícita del usuario: todos los campos heredados y propios se inicializan de la base a la derivada en orden de declaración, y luego solo el constructor seleccionado más cercano se ejecuta una vez. No hay llamadas automáticas a constructores de ancestros ni valores por defecto undefined para campos; la regresión observa 9

Decisión	Razón y límite
Referencias a listas de longitud fija	Reutilizar los tipos de lista semánticos y preservar la mutación a través de alias; foreach congela la referencia y la longitud iniciales, crea enlaces mutables por elemento y usa la pila de transferencias existente
Decodificación HTTP estricta	Compartir las guardas de transporte de Analyze: JSON/UTF-8 válido, `source` exacto y único, sin datos sobrantes, límite de 1 MiB y método/tipo de medio correctos
Texto TAC generado en el servidor	`Program.String()` sirve a la CLI y a HTTP; el navegador solo representa los resultados, lo que evita duplicar el formato
Guarda ante bloque de ciclo recuperado faltante	Preservar la recuperación sintáctica ubicada existente, no una captura genérica de panic ni un cuerpo vacío inventado
Valores desconocidos etiquetados, no semántica nueva	Reutilizar las identidades semánticas y los marcos existentes; validar los operandos reales y las fronteras tipadas en lugar de cambiar Analyze o inventar valores por defecto
Manejadores explícitos en la pila dinámica de llamadas	Preservar los efectos anteriores a un fallo del lenguaje, desenrollar hasta el manejador activo más cercano y quitarlo antes de entrar al cuerpo de catch; los closures léxicos no retienen manejadores
Verificar el retorno antes de la limpieza	Evaluar y verificar el tipo del valor de retorno mientras su manejador está activo; return/break/continue que escapan quitan solo los manejadores atravesados
Print acotado y categorías de fallo	Texto determinista y fallos del lenguaje explícitos y capturables; el IR mal formado, los límites de recursos y los fallos del anfitrión siguen siendo fatales, sin recuperación genérica

Alcance del resultado

Las guardas dinámicas de firma e inicialización, null, campos y límites de listas son obligaciones del backend ejercitadas por el oráculo de pruebas, no promesas estáticas más fuertes de Analyze. Un programa inválido en tiempo de ejecución pero aceptado por Analyze puede generar TAC protegido y fallar al ejecutarse en las pruebas. Los errores sintácticos y semánticos siguen produciendo TAC nil y sin codegen; las guardas estáticas conservadoras de inicialización siguen siendo explícitas. No se afirma contar con ensamblador, ABI física, recolector de basura ni ejecutor de producto. La brecha inicial de cobertura está resuelta, y aquí no se establece ninguna brecha normativa concreta nueva. La cobertura de construcciones no equivale a corrección exhaustiva ni a certificación visual en el navegador.